

THE COMPLETE DEVELOPER'S HANDBOOK

PostgreSQL & SQL

Zero to Mastery

A deep, example-driven guide covering SQL fundamentals, PostgreSQL internals, query optimization, and 100+ real interview questions with detailed answers — built for developers who want to actually understand the "why," not just memorize syntax.

SQL Fundamentals

Joins & Subqueries

Indexing & Performance

PL/pgSQL

Interview Prep

Table of Contents

Part I — Foundations

1. What is a Database, RDBMS & SQL
2. Installing & Connecting to PostgreSQL
3. Databases, Schemas, Tables & Data Types
4. DDL — Creating & Altering Structures
5. Constraints — Enforcing Data Integrity
6. DML — Insert, Update, Delete

Part II — Querying Data

7. SELECT, Filtering, Sorting & Pagination
8. Operators & Pattern Matching
9. Aggregate Functions, GROUP BY & HAVING
10. Joins — Inner, Outer, Cross, Self
11. Subqueries & Common Table Expressions (CTEs)
12. Window Functions
13. Set Operations — UNION, INTERSECT, EXCEPT

Part III — Database Design

14. Normalization (1NF → BCNF)
15. Keys, Relationships & ER Modeling
16. Indexes — Types & Strategy
17. Views & Materialized Views

Part IV — Transactions & Programmability

18. Transactions, ACID & Isolation Levels
19. Locking & Concurrency Control
20. Functions & PL/pgSQL
21. Triggers

Part V — PostgreSQL-Specific Mastery

22. JSON & JSONB
23. Arrays & Composite Types
24. Full-Text Search
25. Partitioning Large Tables
26. Query Performance — EXPLAIN ANALYZE
27. Users, Roles & Security
28. Backup, Restore & Extensions

Part VI — Interview Preparation

29. Conceptual & Theory Questions (Beginner → Advanced)

30. Query Writing Round — Practice Problems with Solutions

31. Query Optimization & Scenario-Based Questions

32. Rapid-Fire Cheat Sheet

1. What is a Database, RDBMS & SQL

1.1 The Core Idea

A **database** is an organized collection of structured data stored electronically so it can be efficiently created, read, updated, and deleted (the "CRUD" operations). A **Database Management System (DBMS)** is the software that manages that data — handling storage, retrieval, concurrency, and security. A **Relational Database Management System (RDBMS)** is a DBMS built on the *relational model*, proposed by Edgar F. Codd in 1970, where data is organized into **tables (relations)** made of rows (tuples) and columns (attributes), and relationships between tables are expressed through shared key values rather than physical pointers.

SQL (Structured Query Language) is the standard language used to communicate with an RDBMS. It is declarative — you describe *what* data you want, not *how* to fetch it; the database's query planner decides the "how."

1.2 Why PostgreSQL?

PostgreSQL ("Postgres") is a free, open-source, object-relational database that has been in continuous development since 1986. It is prized in the industry for:

- **Standards compliance** — one of the most SQL-standard-compliant databases available.
- **Extensibility** — custom data types, operators, functions, and even procedural languages (PL/pgSQL, PL/Python, PL/v8).
- **Advanced data types** — native JSON/JSONB, arrays, ranges, geometric types, UUID, hstore.
- **MVCC (Multi-Version Concurrency Control)** — readers never block writers and vice versa.
- **Robustness** — full ACID compliance, sophisticated indexing (B-tree, GIN, GiST, BRIN, Hash), and mature replication.

1.3 SQL Sub-Languages

Category	Full Form	Commands	Purpose
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE	Defines/modifies structure (schema)
DML	Data Manipulation Language	INSERT, UPDATE, DELETE	Modifies the actual data
DQL	Data Query Language	SELECT	Retrieves data
DCL	Data Control Language	GRANT, REVOKE	Controls access/permissions
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manages transactions

Interview Q: What's the difference between SQL and a database like PostgreSQL?

SQL is a *language* — a specification for querying and manipulating relational data. PostgreSQL is a specific *RDBMS product* that implements SQL (plus its own extensions). Comparing them is like comparing "English" to "a particular book" — one is the language, the other is a piece of software that speaks it.

2. Installing & Connecting to PostgreSQL

2.1 Installation Overview

On Ubuntu/Debian: `sudo apt install postgresql postgresql-contrib` . On macOS: `brew install postgresql@16` . On Windows, use the official installer from [postgresql.org](https://www.postgresql.org), which bundles pgAdmin (a GUI). For quick experimentation, Docker is the cleanest option:

```
docker run --name pg-dev -e POSTGRES_PASSWORD=secret \  
-p 5432:5432 -d postgres:16
```

2.2 Connecting

```
# Connect via the psql CLI  
psql -h localhost -U postgres -d mydatabase  
  
# Common psql meta-commands  
\l          -- list all databases  
\c dbname  -- connect to a database  
\dt        -- list tables in current schema  
\d table    -- describe a table's structure  
\du        -- list roles/users  
\q         -- quit
```

Note: PostgreSQL's default port is `5432` . The default administrative role is usually `postgres` .

3. Databases, Schemas, Tables & Data Types

3.1 The Hierarchy

A single PostgreSQL **server (cluster)** can host multiple **databases**. Each database contains one or more **schemas** (namespaces — the default is `public`). Each schema contains **tables, views, functions, indexes**, etc. This gives you: `server → database → schema → table → row/column`.

```
CREATE DATABASE company_db;
CREATE SCHEMA hr;
CREATE TABLE hr.employees (...); -- schema-qualified table name
```

3.2 Core PostgreSQL Data Types

Category	Types	Notes
Numeric	SMALLINT, INTEGER, BIGINT, DECIMAL/NUMERIC, REAL, DOUBLE PRECISION, SERIAL	NUMERIC is exact (for money); REAL/DOUBLE are approximate floating point
Character	CHAR(n), VARCHAR(n), TEXT	TEXT has no length limit and is preferred in Postgres — VARCHAR(n) has no performance benefit over TEXT here
Boolean	BOOLEAN	TRUE/FALSE/NULL
Date/Time	DATE, TIME, TIMESTAMP, TIMESTAMPTZ, INTERVAL	Always prefer TIMESTAMPTZ (timezone-aware) for anything user-facing
UUID	UUID	Common for primary keys in distributed systems
JSON	JSON, JSONB	JSONB is binary, indexable, and almost always preferred
Array	INTEGER[], TEXT[], etc.	Native array columns — unusual among RDBMSs
Other	ENUM, RANGE types, GEOMETRY (via PostGIS), hstore	Extensible via CREATE TYPE / extensions

Interview Q: SERIAL vs IDENTITY — which should you use for auto-incrementing keys?

`SERIAL` is a legacy Postgres convenience that creates a backing sequence implicitly. The SQL-standard, modern approach is `GENERATED ALWAYS AS IDENTITY`, which behaves more predictably (you can't accidentally insert a conflicting value) and is the recommended approach in Postgres 10+.

```
-- Legacy
id SERIAL PRIMARY KEY

-- Modern, preferred
id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY
```

4. DDL — Creating & Altering Structures

4.1 CREATE TABLE

```
CREATE TABLE employees (  
    employee_id    INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    first_name     VARCHAR(50) NOT NULL,  
    last_name      VARCHAR(50) NOT NULL,  
    email          TEXT UNIQUE NOT NULL,  
    salary         NUMERIC(10,2) CHECK (salary > 0),  
    department_id  INTEGER REFERENCES departments(department_id),  
    hire_date      DATE NOT NULL DEFAULT CURRENT_DATE,  
    created_at     TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

4.2 ALTER TABLE

```
ALTER TABLE employees ADD COLUMN phone VARCHAR(15);  
ALTER TABLE employees DROP COLUMN phone;  
ALTER TABLE employees ALTER COLUMN salary SET NOT NULL;  
ALTER TABLE employees RENAME COLUMN email TO email_address;  
ALTER TABLE employees RENAME TO staff;  
ALTER TABLE employees ADD CONSTRAINT chk_salary CHECK (salary >= 0);
```

4.3 DROP vs TRUNCATE vs DELETE

Command	Removes	Rollback-able?	Resets identity?	Speed
DELETE FROM t	Rows (optionally filtered by WHERE)	Yes (in a transaction)	No	Slow (row-by-row, logged, fires triggers)
TRUNCATE t	All rows	Yes in Postgres (unlike some DBs)	Yes	Fast (deallocates pages)
DROP TABLE t	The entire table structure + data	Yes (in a transaction)	N/A — table gone	Instant

Interview Q: Why is TRUNCATE faster than an unfiltered DELETE?

DELETE scans and removes rows one at a time, generates a row in the WAL (write-ahead log) for each, and fires any per-row triggers, leaving dead tuples for VACUUM to clean up. TRUNCATE deallocates the data pages directly without scanning them and logs only the page deallocations — dramatically cheaper for wiping an entire table.

5. Constraints — Enforcing Data Integrity

Constraint	Purpose	Example
PRIMARY KEY	Uniquely identifies each row; implies NOT NULL + UNIQUE	<code>id INT PRIMARY KEY</code>
FOREIGN KEY	Enforces referential integrity to another table	<code>dept_id INT REFERENCES departments(id)</code>
UNIQUE	No two rows can share this value	<code>email TEXT UNIQUE</code>
NOT NULL	Column must always have a value	<code>name TEXT NOT NULL</code>
CHECK	Custom boolean condition each row must satisfy	<code>CHECK (age >= 18)</code>
DEFAULT	Value used when none is supplied	<code>status TEXT DEFAULT 'active'</code>

5.1 Foreign Key Actions

```
CREATE TABLE orders (  
  order_id  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  customer_id INTEGER REFERENCES customers(id)  
    ON DELETE CASCADE      -- delete orders when the customer is deleted  
    ON UPDATE CASCADE      -- propagate PK changes  
);
```

Other options: `ON DELETE SET NULL`, `ON DELETE RESTRICT` (default — blocks the delete if children exist), `ON DELETE NO ACTION` (like `RESTRICT` but checked at end of transaction).

Interview Q: Can a table have multiple UNIQUE constraints but only one PRIMARY KEY? Why?

Yes. A table can have exactly one `PRIMARY KEY` (it defines the table's canonical identity and is what foreign keys typically reference), but any number of `UNIQUE` constraints on other columns (e.g., an email or username) that also can't repeat, but aren't the row's "official" identifier. Unlike `PRIMARY KEY`, `UNIQUE` columns can accept `NULL`s (multiple `NULL`s are allowed since `NULL` is never considered equal to `NULL`).

6. DML — Insert, Update, Delete

6.1 INSERT

```
-- Single row
INSERT INTO employees (first_name, last_name, email, salary, department_id)
VALUES ('Asha', 'Rao', 'asha@company.com', 65000, 3);

-- Multiple rows in one statement (fast – single round trip)
INSERT INTO employees (first_name, last_name, email, salary, department_id)
VALUES
    ('Vikram', 'Shetty', 'vikram@company.com', 58000, 2),
    ('Meera', 'Nair', 'meera@company.com', 72000, 1);

-- Insert from a query
INSERT INTO archived_employees SELECT * FROM employees WHERE hire_date < '2015-01-01';

-- Upsert: insert, or update if a conflict occurs
INSERT INTO employees (email, salary) VALUES ('asha@company.com', 70000)
ON CONFLICT (email) DO UPDATE SET salary = EXCLUDED.salary;
```

6.2 UPDATE

```
UPDATE employees
SET salary = salary * 1.10
WHERE department_id = 3;

-- Update using data from another table
UPDATE employees e
SET salary = s.new_salary
FROM salary_revisions s
WHERE e.employee_id = s.employee_id;
```

6.3 DELETE

```
DELETE FROM employees WHERE hire_date < '2010-01-01';

-- Delete using a join-like condition
DELETE FROM orders o
USING customers c
WHERE o.customer_id = c.id AND c.is_banned = true;
```

Common mistake: Running `UPDATE` or `DELETE` without a `WHERE` clause affects *every row in the table*. Always run a `SELECT` with the same `WHERE` clause first to verify which rows will be touched.

Interview Q: What does `ON CONFLICT DO UPDATE` (upsert) solve, and what came before it?

Before Postgres 9.5, achieving "insert or update" atomically required a manual pattern: try `UPDATE`, check row count, `INSERT` if zero rows affected, all wrapped carefully to avoid race conditions under concurrent access. `INSERT ... ON CONFLICT` does this atomically in one statement, referencing the would-be-inserted row via the special `EXCLUDED` alias, removing the race condition and reducing round trips.

7. SELECT, Filtering, Sorting & Pagination

7.1 Anatomy of a SELECT (Logical Order of Execution)

SQL is written in one order but **executed** in a different logical order. Understanding this order is one of the most valuable mental models for both writing and debugging queries:

```
FROM -> JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> LIMIT/OFFSET
```

This is why you can't reference a column alias defined in SELECT inside a WHERE clause (WHERE runs before SELECT), but you *can* use it in ORDER BY (which runs after SELECT).

```
SELECT department_id, COUNT(*) AS emp_count
FROM employees
WHERE salary > 40000
GROUP BY department_id
HAVING COUNT(*) > 5
ORDER BY emp_count DESC
LIMIT 10;
```

7.2 Pagination

```
SELECT * FROM employees ORDER BY employee_id LIMIT 20 OFFSET 40; -- page 3, 20/page

-- Keyset (cursor) pagination -- far more efficient at scale than OFFSET
SELECT * FROM employees
WHERE employee_id > 140 -- last id seen on the previous page
ORDER BY employee_id
LIMIT 20;
```

Why keyset pagination matters: `OFFSET 100000` forces Postgres to scan and discard 100,000 rows before returning results — it gets linearly slower as pages grow. Keyset pagination uses an indexed WHERE condition, so every page is equally fast.

8. Operators & Pattern Matching

Operator	Meaning	Example
=, !=/<>, >, <, >=, <=	Comparison	WHERE age >= 18
BETWEEN	Inclusive range	WHERE salary BETWEEN 30000 AND 60000
IN	Match against a list	WHERE dept_id IN (1,2,3)
LIKE / ILIKE	Pattern match (ILIKE = case-insensitive)	WHERE name ILIKE 'a%'
IS NULL / IS NOT NULL	Null checks (never use = NULL)	WHERE manager_id IS NULL
AND / OR / NOT	Boolean logic	WHERE active = true AND age > 18
~ / ~*	Regex match (case-sensitive / insensitive) — Postgres extension	WHERE email ~ '^[a-z]+@'

Interview Q: Why does WHERE salary = NULL never return rows?

NULL represents "unknown," not a value. Any comparison against NULL (including `= NULL`) evaluates to NULL (neither true nor false), and WHERE only keeps rows where the condition is TRUE. You must use `IS NULL` / `IS NOT NULL`, which are special constructs designed to test for unknown-ness directly.

9. Aggregate Functions, GROUP BY & HAVING

9.1 The Aggregate Toolkit

Function	Description
COUNT(*), COUNT(col)	Row count; COUNT(col) skips NULLs, COUNT(*) doesn't
SUM(col), AVG(col)	Total / mean (NULLs ignored)
MIN(col), MAX(col)	Extremes
STRING_AGG(col, ', ')	Concatenate group values into one string
ARRAY_AGG(col)	Collect group values into a Postgres array

9.2 GROUP BY vs HAVING vs WHERE

```
SELECT department_id, AVG(salary) AS avg_sal
FROM employees
WHERE hire_date > '2018-01-01'  -- filters rows BEFORE grouping
GROUP BY department_id
HAVING AVG(salary) > 50000;      -- filters GROUPS after aggregation
```

The core rule: **WHERE cannot use aggregate functions** (rows aren't grouped yet at that stage); **HAVING is specifically for filtering on aggregate results**. Every non-aggregated column in SELECT must appear in GROUP BY.

Interview Q: Difference between WHERE and HAVING?

WHERE filters individual rows before any grouping/aggregation happens; it cannot reference aggregate functions. HAVING filters entire groups after GROUP BY has run aggregate calculations; it's specifically meant to test aggregate conditions like `HAVING COUNT(*) > 5`. Using WHERE where you mean HAVING (or vice-versa) is one of the most common SQL interview traps.

10. Joins — Inner, Outer, Cross, Self

10.1 Sample Schema for This Chapter

```
employees(employee_id, name, department_id, manager_id)
departments(department_id, department_name)
```

10.2 INNER JOIN

Returns only rows that have a match in *both* tables.

```
SELECT e.name, d.department_name
FROM employees e
INNER JOIN departments d ON e.department_id = d.department_id;
```

10.3 LEFT (OUTER) JOIN

Returns all rows from the left table, plus matching rows from the right; unmatched right-side columns come back as NULL. This is the most-used join type for "show me everything, including things with no related record" (e.g. every employee, even ones without a department).

```
SELECT e.name, d.department_name
FROM employees e
LEFT JOIN departments d ON e.department_id = d.department_id;
```

10.4 RIGHT JOIN & FULL OUTER JOIN

```
-- RIGHT JOIN: all departments, even ones with zero employees
SELECT e.name, d.department_name
FROM employees e
RIGHT JOIN departments d ON e.department_id = d.department_id;

-- FULL OUTER JOIN: everything from both sides, matched where possible
SELECT e.name, d.department_name
FROM employees e
FULL OUTER JOIN departments d ON e.department_id = d.department_id;
```

10.5 CROSS JOIN

Produces the Cartesian product — every row of table A paired with every row of table B. Rarely used directly except for generating combinations (e.g., all size × color variants of a product).

```
SELECT s.size, c.color FROM sizes s CROSS JOIN colors c;
```

10.6 SELF JOIN

A table joined to itself — the classic use case is hierarchical data, like an employee-to-manager relationship stored in one table.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id;
```

10.7 Visual Summary

Join Type	Returns
INNER JOIN	Only matching rows in both tables
LEFT JOIN	All of left + matches from right (NULL if none)
RIGHT JOIN	All of right + matches from left (NULL if none)
FULL OUTER JOIN	All rows from both, matched where possible
CROSS JOIN	Every combination of rows (Cartesian product)
SELF JOIN	A table joined to itself, using aliases

Interview Q: How do you find employees who have NO department assigned, using a join?

Use a LEFT JOIN from employees to departments, then filter for rows where the department side is NULL — meaning no match was found:

```
SELECT e.name FROM employees e LEFT JOIN departments d ON e.department_id = d.department_id WHERE
d.department_id IS NULL;
```

This "anti-join" pattern (LEFT JOIN + IS NULL) is extremely common in interviews.

Interview Q: What's the difference between WHERE and ON when using LEFT JOIN?

Conditions in **ON** are applied *while performing the join* — for a LEFT JOIN, non-matching right-side rows are still kept (with NULLs) even if the ON condition fails. Conditions in **WHERE** are applied *after* the join is complete, so filtering on a right-table column in WHERE can inadvertently convert your LEFT JOIN into behaving like an INNER JOIN (because rows with NULL fail most WHERE conditions and get dropped).

11. Subqueries & Common Table Expressions (CTEs)

11.1 Types of Subqueries

```
-- Scalar subquery (returns a single value) -- usable anywhere an expression is
SELECT name, salary,
       (SELECT AVG(salary) FROM employees) AS company_avg
FROM employees;

-- Subquery in WHERE with IN
SELECT name FROM employees
WHERE department_id IN (SELECT department_id FROM departments WHERE location = 'Bangalore');

-- Correlated subquery -- re-evaluated per outer row, references the outer query
SELECT e.name, e.salary
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary) FROM employees e2 WHERE e2.department_id = e.department_id
);

-- EXISTS -- often faster than IN for large subqueries; stops at first match
SELECT d.department_name
FROM departments d
WHERE EXISTS (SELECT 1 FROM employees e WHERE e.department_id = d.department_id);
```

11.2 Common Table Expressions (WITH clause)

A CTE names a subquery so it can be referenced (even multiple times) later in the statement — improving readability over deeply nested subqueries.

```
WITH dept_avg AS (
    SELECT department_id, AVG(salary) AS avg_sal
    FROM employees
    GROUP BY department_id
)
SELECT e.name, e.salary, d.avg_sal
FROM employees e
JOIN dept_avg d ON e.department_id = d.department_id
WHERE e.salary > d.avg_sal;
```

11.3 Recursive CTEs

Used for hierarchical/graph-like data — e.g., traversing an org chart or a category tree.

```

WITH RECURSIVE org_chart AS (
  -- Anchor: the top-level (CEO, no manager)
  SELECT employee_id, name, manager_id, 1 AS level
  FROM employees WHERE manager_id IS NULL

  UNION ALL

  -- Recursive step: find employees reporting to the previous level
  SELECT e.employee_id, e.name, e.manager_id, oc.level + 1
  FROM employees e
  JOIN org_chart oc ON e.manager_id = oc.employee_id
)
SELECT * FROM org_chart ORDER BY level;

```

Interview Q: IN vs EXISTS vs JOIN — when would you pick each?

JOIN is generally fastest when you need columns from both tables and the join keys are indexed — the planner can use efficient hash/merge join algorithms. **EXISTS** is ideal for existence checks against large subqueries because it short-circuits at the first match and doesn't materialize the full subquery result. **IN** is fine for small, static lists but can perform poorly with large subquery result sets, and its NULL-handling is a classic gotcha (**NOT IN** with a subquery that returns even one NULL will return zero rows, unexpectedly) — **NOT EXISTS** is the safer choice for negation.

12. Window Functions

12.1 What Makes Window Functions Different

Unlike GROUP BY (which collapses rows into one row per group), a window function computes a value *across a set of related rows* ("the window") while still returning **one output row per input row**. This is what lets you compute a running total, a rank, or "compare this row to the previous row" without losing row-level detail.

```
SELECT name, department_id, salary,
       RANK()      OVER (PARTITION BY department_id ORDER BY salary DESC) AS dept_rank,
       ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) AS row_num,
       SUM(salary) OVER (PARTITION BY department_id) AS dept_total,
       LAG(salary) OVER (PARTITION BY department_id ORDER BY salary DESC) AS prev_salary
FROM employees;
```

12.2 Key Window Functions

Function	Purpose
ROW_NUMBER()	Unique sequential number per row within the partition; no ties
RANK()	Same rank for ties, then skips the next rank(s) (1,2,2,4)
DENSE_RANK()	Same rank for ties, but no gap afterward (1,2,2,3)
NTILE(n)	Splits rows into n roughly equal buckets (e.g. quartiles)
LAG(col, n)	Value from n rows before the current row
LEAD(col, n)	Value from n rows after the current row
FIRST_VALUE / LAST_VALUE	First/last value in the window frame
SUM/AVG/COUNT() OVER(...)	Running or partition-wide aggregates without collapsing rows

12.3 Classic Use Case: Top-N per Group

```
-- Find the highest-paid employee in EACH department
SELECT * FROM (
    SELECT name, department_id, salary,
           ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rn
    FROM employees
) ranked
WHERE rn = 1;
```

Interview Q: RANK() vs DENSE_RANK() vs ROW_NUMBER() — explain with a tie.

Given salaries [90k, 90k, 80k]: **ROW_NUMBER()** gives 1, 2, 3 (arbitrarily breaks the tie, no repeats). **RANK()** gives 1, 1, 3 (ties share a rank, but the next rank skips ahead to account for the tied rows). **DENSE_RANK()** gives 1, 1, 2 (ties share a rank, and the next distinct value continues immediately with no gap). This exact scenario is one of the most frequently asked SQL interview questions.

13. Set Operations — UNION, INTERSECT, EXCEPT

```
-- UNION removes duplicates; UNION ALL keeps them (and is faster)
SELECT name FROM current_employees
UNION
SELECT name FROM contractors;

-- INTERSECT: rows present in BOTH result sets
SELECT product_id FROM this_month_sales
INTERSECT
SELECT product_id FROM last_month_sales;

-- EXCEPT: rows in the first query but NOT in the second
SELECT customer_id FROM all_customers
EXCEPT
SELECT customer_id FROM active_subscribers;
```

Rule: all queries combined with a set operator must have the same number of columns, in compatible data types, in the same order. Column names in the result come from the first query.

Interview Q: UNION vs UNION ALL — which should you default to?

Default to **UNION ALL** unless you specifically need duplicates removed. **UNION** has to sort/hash the entire combined result set to detect and remove duplicates, which is significantly more expensive. If you already know the two sets are disjoint (e.g., partitioned by a flag), **UNION ALL** is strictly better.

14. Normalization (1NF → BCNF)

Normalization is the process of structuring tables to reduce data redundancy and prevent update/insert/delete anomalies, by progressively enforcing stricter rules called "normal forms."

14.1 First Normal Form (1NF)

Every column must hold atomic (indivisible) values, and each row must be unique. A column storing "Math, Physics, Chemistry" as one comma-separated string violates 1NF — it should be split into a separate row per subject (or a related table).

14.2 Second Normal Form (2NF)

Must be in 1NF, and every non-key column must depend on the *entire* primary key — not just part of it. This only matters for tables with a composite primary key. Example: in `order_items(order_id, product_id, product_name, quantity)`, `product_name` depends only on `product_id`, not on the full `(order_id, product_id)` key — a 2NF violation ("partial dependency"). Fix: move `product_name` to a `products` table.

14.3 Third Normal Form (3NF)

Must be in 2NF, and no non-key column may depend on another non-key column ("transitive dependency"). Example: `employees(emp_id, dept_id, dept_location)` — `dept_location` depends on `dept_id`, not directly on `emp_id`. Fix: move `dept_location` into a `departments` table.

14.4 Boyce-Codd Normal Form (BCNF)

A stricter version of 3NF: for every functional dependency $X \rightarrow Y$, X must be a superkey. BCNF resolves rare edge cases 3NF misses, typically involving overlapping composite candidate keys.

Form	Rule
1NF	Atomic values, unique rows, no repeating groups
2NF	1NF + no partial dependency on part of a composite key
3NF	2NF + no transitive dependency between non-key columns
BCNF	3NF + every determinant is a candidate key

Interview Q: Why would you ever denormalize a database on purpose?

Normalization optimizes for data integrity and minimal redundancy, but every join has a query-time cost. In read-heavy systems (analytics/reporting/dashboards), controlled denormalization — duplicating some data or pre-aggregating it — trades a bit of redundancy for significantly faster reads, since you avoid expensive joins. The classic answer: "normalize until it hurts, denormalize until it works," balancing integrity against real performance needs.

15. Keys, Relationships & ER Modeling

Key Type	Description
Candidate Key	Any column (or set) that could uniquely identify a row
Primary Key	The chosen candidate key; only one per table
Alternate Key	Candidate keys not chosen as primary (often enforced via UNIQUE)
Composite Key	A primary key made of 2+ columns together
Foreign Key	A column referencing another table's primary key
Surrogate Key	An artificial key (e.g. auto-increment ID) with no business meaning
Natural Key	A key derived from real-world data (e.g. an email or SSN)

15.1 Relationship Cardinalities

- **One-to-Many** — most common; a foreign key on the "many" side (e.g., one department has many employees).
- **Many-to-Many** — requires a junction/bridge table with two foreign keys (e.g., students $\leftrightarrow$ courses via an `enrollments` table).
- **One-to-One** — rare; usually modeled by putting a UNIQUE foreign key on one side, often used to split a wide table for security or performance reasons.

Interview Q: Surrogate key vs natural key — which do you prefer for a primary key, and why?

Surrogate keys (auto-increment integers or UUIDs) are generally preferred because natural keys can change (a person's email or phone number), can be composite and unwieldy for foreign keys, and may not be as compact/performant for indexing. The trade-off: surrogate keys still need a UNIQUE constraint on the natural key to prevent duplicate real-world entities.

16. Indexes — Types & Strategy

16.1 Why Indexes Exist

Without an index, PostgreSQL must perform a **sequential scan** — reading every row in the table to find matches. An index is a separate, sorted data structure that lets the database jump directly to matching rows, similar to a book's index letting you skip straight to a topic instead of reading every page.

```
CREATE INDEX idx_employees_dept ON employees(department_id);
CREATE UNIQUE INDEX idx_employees_email ON employees(email);
CREATE INDEX idx_employees_name_dept ON employees(last_name, department_id); -- composite
```

16.2 Index Types in PostgreSQL

Type	Best For
B-tree (default)	Equality & range queries (=, <, >, BETWEEN, ORDER BY) — the workhorse index
Hash	Pure equality (=) lookups only; rarely chosen over B-tree today
GIN	Composite/multi-valued data — JSONB, arrays, full-text search
GiST	Geometric data, ranges, nearest-neighbor search (used heavily by PostGIS)
BRIN	Very large, naturally-ordered tables (e.g., time-series data by timestamp) — tiny, cheap index

16.3 The Cost of Indexes

Indexes speed up reads but slow down writes (INSERT/UPDATE/DELETE must also update every index on the table) and consume disk space. The rule of thumb: index columns that are frequently filtered, joined, or sorted on — not every column.

Interview Q: You added an index but the query is still slow. What could be wrong?

Common causes: (1) The planner chose a sequential scan anyway because the table is small or the filter matches a large fraction of rows (an index isn't always faster). (2) A function or type cast was applied to the indexed column in the WHERE clause (e.g. `WHERE LOWER(email) = ...`) — this needs a matching expression index. (3) The column is the second column in a composite index but the query doesn't filter on the first column (B-tree composite indexes are ordered left-to-right, like a phone book sorted by last-then-first name). (4) Table statistics are stale — run `ANALYZE`. Always confirm with `EXPLAIN ANALYZE` rather than guessing.

17. Views & Materialized Views

```
-- Regular view: a stored, reusable query. Always reflects live data.
CREATE VIEW high_earners AS
SELECT name, salary, department_id FROM employees WHERE salary > 80000;

SELECT * FROM high_earners WHERE department_id = 2; -- query it like a table

-- Materialized view: physically stores the result set; must be refreshed manually
CREATE MATERIALIZED VIEW dept_summary AS
SELECT department_id, COUNT(*) AS headcount, AVG(salary) AS avg_salary
FROM employees GROUP BY department_id;

REFRESH MATERIALIZED VIEW dept_summary;
```

Interview Q: View vs Materialized View?

A regular **view** is just a saved query — it has no storage of its own and always re-executes against live data, so it's always up-to-date but costs the same as running the underlying query each time. A **materialized view** actually stores the computed result set on disk, so reads are fast (like reading a table), but the data is a snapshot that goes stale until you explicitly run `REFRESH MATERIALIZED VIEW`. Materialized views suit expensive aggregations that don't need to be real-time (dashboards, reports).

18. Transactions, ACID & Isolation Levels

18.1 ACID Properties

Property	Meaning
Atomicity	A transaction's operations all succeed, or all fail together — no partial application
Consistency	A transaction moves the database from one valid state to another, respecting all constraints
Isolation	Concurrent transactions don't interfere with each other's intermediate states
Durability	Once committed, data survives even a crash immediately after (via the WAL)

```

BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT;  -- or ROLLBACK; if anything looks wrong

-- Savepoints allow partial rollback within a transaction
BEGIN;
INSERT INTO logs (msg) VALUES ('step 1');
SAVEPOINT sp1;
INSERT INTO logs (msg) VALUES ('step 2 - risky');
ROLLBACK TO sp1;  -- undoes step 2 only, step 1 remains
COMMIT;

```

18.2 Isolation Levels & Concurrency Anomalies

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
READ UNCOMMITTED*	Possible	Possible	Possible
READ COMMITTED (Postgres default)	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Prevented in Postgres**
SERIALIZABLE	Prevented	Prevented	Prevented

*Postgres treats READ UNCOMMITTED identically to READ COMMITTED — it has no true dirty reads. **Postgres's REPEATABLE READ is implemented via MVCC snapshots and also prevents phantom reads, stricter than the SQL standard requires.

- **Dirty read** — reading uncommitted changes from another transaction.
- **Non-repeatable read** — re-reading the same row within a transaction gives a different value because another transaction committed a change in between.
- **Phantom read** — re-running the same query returns a different *set of rows* because another transaction inserted/deleted matching rows.

Interview Q: What isolation level does PostgreSQL use by default, and why is that a reasonable default?

READ COMMITTED — each statement (not the whole transaction) sees a fresh snapshot of committed data at the moment it runs. It balances good concurrency (transactions rarely block each other) against protection from dirty reads, which is enough correctness for the vast majority of applications. Stricter levels like **SERIALIZABLE** add real overhead (retries on conflict) and are reserved for scenarios with strict invariants, like double-booking prevention or financial ledgers.

19. Locking & Concurrency Control

PostgreSQL relies primarily on **MVCC (Multi-Version Concurrency Control)**: rather than locking rows for reads, each transaction sees a consistent "snapshot" of the data as of when it started (or each statement, under `READ COMMITTED`), and writes create new row versions rather than overwriting in place. This is why **readers never block writers, and writers never block readers** in Postgres — a major advantage over lock-based concurrency models.

```
-- Row-level lock: prevents others from updating/deleting the selected rows until commit
SELECT * FROM accounts WHERE id = 1 FOR UPDATE;

-- Advisory locks: application-level locks with no relation to any table
SELECT pg_advisory_lock(12345);
```

Old row versions aren't deleted immediately — they become "dead tuples" that `VACUUM` (run automatically by `autovacuum`) reclaims later. This is the trade-off behind MVCC's non-blocking reads.

20. Functions & PL/pgSQL

```
CREATE OR REPLACE FUNCTION get_employee_bonus(emp_salary NUMERIC)
RETURNS NUMERIC AS $$
BEGIN
    IF emp_salary > 100000 THEN
        RETURN emp_salary * 0.15;
    ELSIF emp_salary > 50000 THEN
        RETURN emp_salary * 0.10;
    ELSE
        RETURN emp_salary * 0.05;
    END IF;
END;
$$ LANGUAGE plpgsql;

SELECT name, get_employee_bonus(salary) FROM employees;

-- A function returning a table (set-returning function)
CREATE OR REPLACE FUNCTION employees_in_dept(dept INT)
RETURNS TABLE(name TEXT, salary NUMERIC) AS $$
BEGIN
    RETURN QUERY
        SELECT e.name, e.salary FROM employees e WHERE e.department_id = dept;
END;
$$ LANGUAGE plpgsql;

SELECT * FROM employees_in_dept(3);
```

21. Triggers

A trigger automatically fires a function in response to INSERT, UPDATE, or DELETE — used for auditing, enforcing complex business rules, or maintaining derived data.

```
CREATE OR REPLACE FUNCTION log_salary_change() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.salary <> OLD.salary THEN
        INSERT INTO salary_audit(employee_id, old_salary, new_salary, changed_at)
        VALUES (OLD.employee_id, OLD.salary, NEW.salary, now());
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_salary_audit
AFTER UPDATE ON employees
FOR EACH ROW
EXECUTE FUNCTION log_salary_change();
```

Interview Q: BEFORE vs AFTER trigger — when would you use each?

A **BEFORE** trigger fires before the change is written and can modify **NEW** to alter what actually gets stored (e.g., auto-normalizing an email to lowercase) or raise an exception to reject the operation entirely. An **AFTER** trigger fires once the change is already committed to the row and is used for side effects that shouldn't block or alter the original write — like writing an audit log entry or sending a notification — since the base operation has already succeeded by that point.

22. JSON & JSONB

PostgreSQL supports storing and querying semi-structured JSON directly in a column — a huge advantage when parts of your schema are variable or evolving.

Type	Storage	Indexable?	Preserves key order?
JSON	Stored as exact text	No (must cast/extract to index)	Yes
JSONB	Stored as decomposed binary	Yes (GIN indexes)	No — but faster to query

In almost all real applications, **use JSONB** — the only reason to use plain JSON is if you must preserve exact formatting/key order or whitespace.

```
CREATE TABLE products (  
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  name TEXT,  
  attributes JSONB  
);  
  
INSERT INTO products (name, attributes) VALUES  
('Laptop', '{"ram_gb": 16, "brand": "Dell", "tags": ["electronics", "office"]}');  
  
-- -> returns JSONB, -> returns text  
SELECT name, attributes->'brand' AS brand_json, attributes->>'brand' AS brand_text  
FROM products;  
  
-- Filter on a JSON field  
SELECT * FROM products WHERE attributes->>'brand' = 'Dell';  
  
-- Containment check (uses GIN index efficiently)  
SELECT * FROM products WHERE attributes @> '{"ram_gb": 16}';  
  
-- Index for fast JSONB queries  
CREATE INDEX idx_products_attrs ON products USING GIN (attributes);
```

Interview Q: When would you choose JSONB over a normalized relational schema?

JSONB fits well when the schema is genuinely variable per row (different product categories have entirely different attribute sets), when the structure changes frequently and migrations would be costly, or for storing external API payloads/config blobs you mostly read as a whole. It's a poor fit when you need strong referential integrity, complex cross-record joins/aggregation on nested fields, or when most of your queries hit the same few sub-fields — those cases usually perform and model better as proper columns/tables.

23. Arrays & Composite Types

```
CREATE TABLE posts (  
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    title TEXT,  
    tags TEXT[]  
);  
  
INSERT INTO posts (title, tags) VALUES ('SQL Guide', ARRAY['sql', 'postgres', 'database']);  
  
SELECT * FROM posts WHERE 'postgres' = ANY(tags); -- contains element  
SELECT * FROM posts WHERE tags @> ARRAY['sql']; -- contains (GIN-indexable)  
SELECT unnest(tags) FROM posts WHERE id = 1; -- expand array into rows
```

24. Full-Text Search

PostgreSQL has built-in full-text search — no separate search engine (like Elasticsearch) needed for moderate-scale use cases.

```
ALTER TABLE articles ADD COLUMN search_vector tsvector
    GENERATED ALWAYS AS (to_tsvector('english', title || ' ' || body)) STORED;

CREATE INDEX idx_articles_search ON articles USING GIN (search_vector);

SELECT title FROM articles
WHERE search_vector @@ to_tsquery('english', 'database & performance');
```

`to_tsvector` converts text into normalized lexemes (stemmed, stop-words removed); `to_tsquery` parses a search query the same way, and `@@` tests if the document matches.

25. Partitioning Large Tables

Partitioning splits one logical table into multiple physical pieces, so queries that filter on the partition key only scan relevant partitions ("partition pruning") — critical for tables with hundreds of millions of rows.

```
CREATE TABLE sales (  
    id BIGINT GENERATED ALWAYS AS IDENTITY,  
    sale_date DATE NOT NULL,  
    amount NUMERIC  
) PARTITION BY RANGE (sale_date);  
  
CREATE TABLE sales_2025 PARTITION OF sales  
    FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');  
  
CREATE TABLE sales_2026 PARTITION OF sales  
    FOR VALUES FROM ('2026-01-01') TO ('2027-01-01');  
  
-- Queries filtering by sale_date automatically hit only the relevant partition  
SELECT SUM(amount) FROM sales WHERE sale_date BETWEEN '2026-01-01' AND '2026-03-31';
```

Other strategies: `PARTITION BY LIST` (discrete categories, e.g. by region) and `PARTITION BY HASH` (evenly distribute rows when there's no natural range/list key).

26. Query Performance — EXPLAIN ANALYZE

`EXPLAIN` shows the planner's chosen execution plan without running the query; `EXPLAIN ANALYZE` actually executes it and reports real timings alongside the plan.

```
EXPLAIN ANALYZE
SELECT * FROM employees WHERE department_id = 3 ORDER BY salary DESC;
```

```
Sort (cost=45.32..45.89 rows=228 width=64) (actual time=0.512..0.520 rows=210 loops=1)
  Sort Key: salary DESC
  -> Index Scan using idx_employees_dept on employees
      (cost=0.29..36.55 rows=228 width=64) (actual time=0.045..0.301 rows=210 loops=1)
      Index Cond: (department_id = 3)
Planning Time: 0.183 ms
Execution Time: 0.612 ms
```

26.1 Key Plan Nodes to Recognize

Node	Meaning
Seq Scan	Full table scan — fine for small tables, a red flag on large ones
Index Scan	Uses an index, then fetches matching rows from the table heap
Index Only Scan	Answers entirely from the index without touching the table — fastest
Nested Loop	For each outer row, scans the inner table/index — good for small inputs
Hash Join	Builds a hash table from one side, probes with the other — good for larger, unsorted inputs
Merge Join	Both inputs sorted on the join key, merged in one pass — great when data is already sorted/indexed that way

Interview Q: A query has "cost=..." numbers in EXPLAIN — are those milliseconds?

No — the cost values in plain `EXPLAIN` are arbitrary planner units (roughly calibrated to disk page fetches), used only to compare plans against each other, not real time. To see actual elapsed time, you need `EXPLAIN ANALYZE`, which really executes the query and reports true milliseconds alongside the estimated costs — useful for spotting cases where the planner's row estimates are wildly wrong (a sign statistics are stale, fixable with `ANALYZE table_name`).

27. Users, Roles & Security

```
CREATE ROLE app_readonly LOGIN PASSWORD 'secret';
GRANT CONNECT ON DATABASE company_db TO app_readonly;
GRANT USAGE ON SCHEMA public TO app_readonly;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO app_readonly;

REVOKE INSERT, UPDATE, DELETE ON employees FROM app_readonly;

-- Row-Level Security: restrict which rows a role can see/modify
ALTER TABLE employees ENABLE ROW LEVEL SECURITY;
CREATE POLICY dept_isolation ON employees
    USING (department_id = current_setting('app.current_dept')::INT);
```


28. Backup, Restore & Extensions

```
# Logical backup (portable, human-restorable SQL)
pg_dump -U postgres company_db > backup.sql
psql -U postgres company_db < backup.sql

# Custom-format backup (compressed, supports parallel restore)
pg_dump -Fc company_db > backup.dump
pg_restore -d company_db backup.dump
```

```
-- Popular extensions
CREATE EXTENSION IF NOT EXISTS "uuid-ossf"; -- UUID generation
CREATE EXTENSION IF NOT EXISTS pgcrypto;    -- encryption functions
CREATE EXTENSION IF NOT EXISTS postgis;     -- geospatial data
CREATE EXTENSION IF NOT EXISTS pg_stat_statements; -- query performance tracking
```

Interview Q: `pg_dump` vs a physical/filesystem backup — what's the trade-off?

`pg_dump` produces a *logical* backup — a portable, version-independent representation of the schema and data (as SQL or a custom archive) that can even be restored into a different Postgres major version. It's slower for very large databases and captures a single consistent point-in-time snapshot. A physical backup (e.g. `pg_basebackup` or filesystem-level snapshots + WAL archiving) copies the raw data files, which is faster for huge databases and supports point-in-time recovery, but ties you to matching Postgres versions/architecture.

29. Conceptual & Theory Questions

29.1 Beginner Level

Q1. What is the difference between CHAR, VARCHAR and TEXT?

`CHAR(n)` is fixed-length and pads with spaces up to `n`. `VARCHAR(n)` is variable-length up to a max of `n`. `TEXT` is variable-length with no limit. In PostgreSQL specifically, all three have virtually identical performance internally, so `TEXT` is generally recommended unless a strict length constraint is a genuine business rule.

Q2. What is a primary key, and can it be NULL?

A primary key uniquely identifies each row in a table. It cannot be NULL (NULL means "unknown," which can't uniquely identify anything) and cannot contain duplicate values. A table can have only one primary key, though that key can be composite (multiple columns).

Q3. What's the difference between a primary key and a unique key?

Both enforce uniqueness, but a table can have only one primary key (which also implies NOT NULL) versus multiple unique keys (which can allow one or more NULLs, since `NULL ≠ NULL` for uniqueness purposes). Primary keys are also typically what foreign keys reference.

Q4. What does the DEFAULT keyword do?

Supplies an automatic value for a column when no explicit value is given in an INSERT — e.g., `status TEXT DEFAULT 'pending'` or `created_at TIMESTAMPTZ DEFAULT now()`.

Q5. What is a foreign key, and what problem does it solve?

A foreign key is a column (or set of columns) in one table that references the primary key of another table, enforcing referential integrity — you cannot insert a row pointing to a parent that doesn't exist, and (depending on the ON DELETE rule) the database controls what happens to child rows if the parent is removed.

Q6. Explain the difference between DELETE, TRUNCATE and DROP.

DELETE removes rows (optionally filtered) and can be rolled back, but is slower and keeps the table structure. TRUNCATE removes all rows quickly by deallocating pages and resets identity columns; still rollback-able in Postgres. DROP removes the entire table (structure and data) permanently from the schema.

Q7. What is NULL, and how is it different from zero or an empty string?

NULL represents the *absence* or *unknown* state of a value — it is not a value itself, and never equals anything, including another NULL. Zero (0) and empty string (") are actual, defined values. This is why `NULL = NULL` evaluates to NULL (unknown), not TRUE.

Q8. What is a schema in PostgreSQL?

A namespace within a database that groups related tables, views, functions, etc. — like a folder inside a database. It lets you organize objects and avoid naming collisions (e.g., `sales.customers` vs. `hr.customers` in the same database).

29.2 Intermediate Level

Q9. Explain the ACID properties with a banking example.

Transferring ₹500 from Account A to Account B requires two updates. **Atomicity** ensures both updates happen or neither does (no money vanishes if the system crashes mid-transfer). **Consistency** ensures the total balance across both accounts stays mathematically correct before and after. **Isolation** ensures a concurrent balance check on Account A doesn't see a half-completed transfer. **Durability** ensures that once the transfer is confirmed/committed, it survives even an immediate server crash.

Q10. What is normalization and why is it important?

The process of organizing tables to eliminate redundant data and avoid insert/update/delete anomalies, by progressively applying normal forms (1NF, 2NF, 3NF, BCNF). It matters because redundant data is a source of inconsistency — e.g., updating a customer's address in one row but forgetting a duplicate row elsewhere.

Q11. What is an index, and what is the trade-off of adding one?

An index is an auxiliary data structure (usually a B-tree) that lets the database find rows without scanning the whole table. The trade-off: indexes speed up reads but slow down writes (every INSERT/UPDATE/DELETE must also maintain each index) and consume additional disk space.

Q12. What is a composite index, and does column order matter?

An index built on multiple columns together, e.g. `(last_name, department_id)`. Order matters greatly — a B-tree composite index is sorted first by the leftmost column, then the next, so it can efficiently serve queries filtering on the leftmost column alone or the leftmost + subsequent columns, but generally cannot efficiently serve a query that filters only on the second column.

Q13. What is a deadlock, and how does PostgreSQL handle it?

A deadlock occurs when two transactions each hold a lock the other needs, and both wait indefinitely (Transaction A waits for a lock held by B, while B waits for a lock held by A). PostgreSQL periodically runs deadlock detection and automatically aborts one of the transactions (rolling it back) so the other can proceed, returning a "deadlock detected" error to the aborted transaction's client.

Q14. What is the difference between a clustered and non-clustered index? Does PostgreSQL have clustered indexes?

A clustered index physically orders the table's rows to match the index order (a table can have only one, since rows can only be sorted one way). A non-clustered index is a separate structure with pointers back to the table's rows. PostgreSQL doesn't maintain a "true" clustered index automatically like SQL Server — but the one-time `CLUSTER table USING index` command physically reorders the table once. After further writes, the table drifts out of that order again since Postgres doesn't keep it clustered on an ongoing basis.

Q15. What is a stored procedure, and how does it differ from a function?

Both are reusable, precompiled blocks of SQL/procedural code. In PostgreSQL, functions (`CREATE FUNCTION`) return a value and can be used inside SELECT statements; procedures (`CREATE PROCEDURE` , added in PG11) are invoked with `CALL` , don't need to return a value, and — importantly — can manage their own transactions (COMMIT/ROLLBACK inside the procedure body), which functions cannot do.

29.3 Advanced Level

Q16. Explain MVCC and why PostgreSQL uses it.

Multi-Version Concurrency Control means every row can have multiple physical versions at once. When a transaction updates a row, Postgres doesn't overwrite it in place — it writes a new row version and marks the old one as expired (but not yet removed). Each transaction sees a consistent snapshot of the data as of when it began (or each statement, under READ COMMITTED). This means readers never block writers

and writers never block readers, dramatically improving concurrency compared to systems that rely on read locks. The trade-off is that dead row versions accumulate and must be cleaned up by (auto)VACUUM, and heavy write/update churn can cause table/index bloat if vacuuming falls behind.

Q17. What is query plan caching, and what are prepared statements?

A prepared statement (`PREPARE ... AS SELECT ...`) parses and plans a parameterized query once, then can be executed repeatedly with different parameter values (`EXECUTE`), skipping repeated parse/plan overhead. PostgreSQL's planner may use a "generic plan" (planned once for any parameter values) after enough executions, versus a "custom plan" (re-planned per execution using the actual parameter values, potentially better if data distribution is skewed) — the planner adaptively decides which is cheaper over repeated calls.

Q18. What is a covering index, and why does it matter for performance?

A covering index includes all the columns a query needs (either as key columns or via the `INCLUDE` clause), so the query can be answered entirely from the index without a separate lookup into the table heap — called an "Index Only Scan." This avoids random I/O against the base table, which is often the most expensive part of a lookup.

```
CREATE INDEX idx_covering ON employees(department_id) INCLUDE (name, salary);
-- A query selecting only name/salary filtered by department_id never touches the table
```

Q19. What's the N+1 query problem, and how do you fix it?

A common ORM antipattern: fetching a list of N parent records, then executing one additional query per parent to fetch related child records (N+1 total queries) instead of a single JOIN or a single batched query. Fixes: use a JOIN, use the ORM's eager-loading feature (e.g. Prisma's `include`, Sequelize's `include`), or batch-fetch children with a single `WHERE parent_id IN (...)` query.

Q20. What is a CTE materialization, and has PostgreSQL's behavior changed here?

Before PostgreSQL 12, a CTE (`WITH ...`) was always materialized — executed once as an "optimization fence," fully computed into a temporary result before the outer query could use it, which sometimes blocked the planner from pushing filters into it. From PostgreSQL 12 onward, non-recursive CTEs are inlined by default (treated like a subquery) unless referenced multiple times or explicitly marked, giving the planner more freedom to optimize. You can force the old behavior with `WITH cte AS MATERIALIZED (...)`.

30. Query Writing Round — Practice Problems with Solutions

Assume this schema throughout: `employees(employee_id, name, department_id, manager_id, salary, hire_date)`, `departments(department_id, department_name)`.

P1. Find the second-highest salary in the company.

```
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC
OFFSET 1 LIMIT 1;

-- Or, more robust if there are ties for #1:
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

P2. Find the Nth highest salary (parameterized).

```
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET (N - 1);

-- Window-function version, handling ties correctly:
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) t WHERE rnk = N;
```

P3. Find employees who earn more than their manager.

```
SELECT e.name AS employee, e.salary, m.name AS manager, m.salary AS manager_salary
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id
WHERE e.salary > m.salary;
```

P4. Find duplicate email entries in a customers table.

```
SELECT email, COUNT(*) AS occurrences
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;
```

P5. Delete duplicate rows, keeping only the earliest (lowest id) for each email.

```

DELETE FROM customers c
USING customers c2
WHERE c.email = c2.email AND c.id > c2.id;

-- Equivalent window-function approach:
DELETE FROM customers
WHERE id IN (
    SELECT id FROM (
        SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) AS rn
        FROM customers
    ) t WHERE rn > 1
);

```

P6. Find departments with more than 5 employees, along with the count.

```

SELECT d.department_name, COUNT(e.employee_id) AS emp_count
FROM departments d
JOIN employees e ON e.department_id = d.department_id
GROUP BY d.department_name
HAVING COUNT(e.employee_id) > 5;

```

P7. Find the employee(s) with the highest salary in each department.

```

SELECT name, department_id, salary FROM (
    SELECT name, department_id, salary,
        RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rnk
    FROM employees
) t WHERE rnk = 1;

```

P8. Calculate month-over-month percentage growth in total sales.

```

WITH monthly AS (
    SELECT date_trunc('month', sale_date) AS month, SUM(amount) AS total
    FROM sales GROUP BY 1
)
SELECT month, total,
    LAG(total) OVER (ORDER BY month) AS prev_month,
    ROUND(
        (total - LAG(total) OVER (ORDER BY month))
        / LAG(total) OVER (ORDER BY month) * 100, 2
    ) AS pct_growth
FROM monthly ORDER BY month;

```

P9. Find customers who placed orders in every month of 2026 (a "loyal customer" query).

```
SELECT customer_id
FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 2026
GROUP BY customer_id
HAVING COUNT(DISTINCT EXTRACT(MONTH FROM order_date)) = 12;
```

P10. Write a query for a running total of sales ordered by date.

```
SELECT sale_date, amount,
       SUM(amount) OVER (ORDER BY sale_date
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM sales;
```

P11. Swap the values of column A and column B for all rows, in one statement.

```
UPDATE my_table SET a = b, b = a;
-- PostgreSQL evaluates the entire SET clause against the OLD row values
-- simultaneously, so this correctly swaps both columns in one pass.
```

P12. Pivot rows into columns: total sales per product, split by quarter.

```
SELECT product_id,
       SUM(amount) FILTER (WHERE EXTRACT(QUARTER FROM sale_date) = 1) AS q1,
       SUM(amount) FILTER (WHERE EXTRACT(QUARTER FROM sale_date) = 2) AS q2,
       SUM(amount) FILTER (WHERE EXTRACT(QUARTER FROM sale_date) = 3) AS q3,
       SUM(amount) FILTER (WHERE EXTRACT(QUARTER FROM sale_date) = 4) AS q4
FROM sales
GROUP BY product_id;
-- FILTER is Postgres's clean, standard-SQL way to do conditional aggregation,
-- cleaner than repeated CASE WHEN inside SUM().
```

31. Query Optimization & Scenario-Based Questions

S1. A production dashboard query that used to take 200ms now takes 8 seconds. How do you debug it?

Step-by-step: (1) Run `EXPLAIN ANALYZE` on the live query to see the actual plan and where time is spent. (2) Check if the planner switched from an Index Scan to a Seq Scan — often caused by stale statistics (fix: `ANALYZE table`) or the table growing enough that a previously-selective filter no longer is. (3) Check for table/index bloat from insufficient (auto)VACUUM — `pg_stat_user_tables` shows dead tuple counts. (4) Check for lock contention from a long-running transaction blocking this query. (5) Confirm no one dropped or renamed an index recently. (6) Look at `pg_stat_statements` for a broader picture of what changed in overall query patterns.

S2. You need to add a NOT NULL column with a default to a table with 200 million rows, in production, without long downtime. How?

In older Postgres versions, adding a column with a non-null DEFAULT required rewriting every row (a long exclusive lock). Since PostgreSQL 11, adding a column with a constant DEFAULT is a fast metadata-only change (the default is stored in the catalog, applied lazily on read) — so this specific case is now cheap. For a DEFAULT based on a volatile expression, or for backfilling existing rows individually, the safe pattern is: add the column as nullable first, backfill in small batches (to avoid long locks and huge WAL bursts), then add the NOT NULL constraint afterward (ideally via `NOT NULL NOT VALID + VALIDATE CONSTRAINT`, which validates without a full table lock).

S3. Your application is seeing "too many connections" errors under load. What are the likely causes and fixes?

PostgreSQL's `max_connections` is a hard ceiling, and each connection carries real memory/process overhead. Likely causes: the application isn't using a connection pool (or the pool is misconfigured too large), connections aren't being released after use (leaks), or traffic genuinely exceeds capacity. Fix: introduce a connection pooler (e.g. PgBouncer) in transaction-pooling mode to multiplex many client connections onto fewer actual Postgres connections, audit application code for unclosed connections, and only raise `max_connections` as a last resort since it directly increases memory pressure.

S4. How would you design a schema for a many-to-many "students enroll in courses" relationship, including tracking a grade per enrollment?

Use a junction table rather than trying to store lists in either table: `enrollments(student_id REFERENCES students, course_id REFERENCES courses, grade TEXT, enrolled_at TIMESTAMPTZ, PRIMARY KEY(student_id, course_id))`. The composite primary key naturally prevents duplicate enrollments, and the extra `grade` column demonstrates why junction tables often carry relationship-specific attributes beyond just the two foreign keys.

S5. How do you safely run a schema migration (e.g., renaming a heavily-used column) with zero downtime?

Avoid a single breaking rename. Use an expand-and-contract pattern: (1) add the new column alongside the old one, (2) write to both columns from the application (or backfill with a trigger), (3) backfill historical data in batches, (4) deploy application code that reads from the new column, (5) once fully migrated and verified, drop the old column in a later, separate deploy. This avoids any single deploy where the schema and running application code are out of sync.

32. Rapid-Fire Cheat Sheet

Topic	One-Line Answer
Default Postgres port	5432
Default isolation level	READ COMMITTED
Concurrency model	MVCC (no read locks; snapshot-based)
Fastest way to check a query plan	EXPLAIN ANALYZE
Best index for equality + range	B-tree
Best index for JSONB / arrays	GIN
Case-insensitive LIKE	ILIKE
Set-returning duplicate removal	UNION (vs. UNION ALL keeps dupes)
Auto-increment, modern syntax	GENERATED ALWAYS AS IDENTITY
Insert-or-update in one statement	INSERT ... ON CONFLICT DO UPDATE
Row lock for update	SELECT ... FOR UPDATE
Ranking with gaps on ties	RANK()
Ranking without gaps on ties	DENSE_RANK()
Ranking, no ties ever	ROW_NUMBER()
View that stores data physically	MATERIALIZED VIEW
Command to update planner statistics	ANALYZE table_name;
Command to reclaim dead tuples	VACUUM (usually via autovacuum)

Final tip for interviews: When asked to write a query live, narrate your thinking out loud — state which tables you need, whether you need a JOIN or subquery, and what edge cases (NULLs, duplicates, ties) you're accounting for. Interviewers weigh your reasoning process as much as the final syntax.